

Seguridad y Resiliencia

1. Minimizacion de datos

Filtrado antes de procesar con IA: el nodo **Filter**, inmediatamente despues del Gmail Trigger, descarta cualquier email cuyo remitente contenga linkedin.com, n8n.cloud o anthropic.com (notificaciones de plataforma, no consultas reales). Estos items nunca llegan al nodo de IA ni se guardan en Airtable — se minimiza tanto el dato procesado como el costo de tokens.

```
Filter (condiciones, combinador AND):
  {{ $json.From }} NOT CONTAINS "linkedin.com"
  {{ $json.From }} NOT CONTAINS "n8n.cloud"
  {{ $json.From }} NOT CONTAINS "anthropic.com"
```

Campos recolectados, solo los necesarios: la tabla Leads solo guarda lo indispensable para clasificar y contactar al huesped — nombre, mail, mensaje original y los campos derivados por la IA (perfil de estadia, prioridad, resumen). No se solicitan ni almacenan datos sensibles (documento de identidad, datos de pago, direccion exacta) en ningun punto del flujo.

Registro de errores acotado: el campo Detalle de Registro_Errores guarda unicamente el mensaje de error tecnico (\$json.error.message), no el contenido del mensaje original del huesped — se evita duplicar datos personales en la tabla de incidentes.

2. Rutas de error (Error Handlers)

El flujo protege sus dos puntos de falla externa (llamada a la API de Claude y envio final por Gmail) con onError: continueErrorOutput. Esto significa que un fallo en esos nodos **no corta la ejecucion**: el item se redirige a una rama de error que deja constancia en Airtable en vez de perderse.

Nodo protegido	Disparador del error	Accion ante fallo
Basic LLM Chain (llamada a Claude)	Timeout, error de API, o salida que no cumple el JSON Schema Registry	Se registra en Registro_Errores con Tipo_Error = "Fallo API IA", Nodo: Basic LLM Chain
Send a message (envio final por Gmail al huesped)	Fallo de la API de Gmail al enviar el mensaje aprobado	Se registra en Registro_Errores: Tipo_Error = "Fallo envio Gmail", Nodo: Send a message

En ambos casos el registro queda con **Estado = “Pendiente”**, lo que funciona como una cola de revision manual: nada se pierde silenciosamente, y cada error queda trazado al nodo exacto que fallo.

Validacion defensiva adicional: el nodo **If** que evalua la aprobacion humana usa looseTypeValidation: true sobre {{ \$json.data.approved }}, para que una diferencia de tipo (string vs. boolean) en la respuesta del formulario de aprobacion no rompa el ruteo ni derive el lead por la rama equivocada.

Observacion para la proxima iteracion: el error de Basic LLM Chain se registra actualmente en **dos** nodos en paralelo (Create or update a record3 y Create a record1), ambos escribiendo en Registro_Errores con datos equivalentes. No rompe el flujo, pero genera un registro duplicado por cada fallo de IA; se puede consolidar en un unico nodo en la proxima version.

3. Human-in-the-loop (HITL)

El punto de control humano obligatorio esta en el nodo **“Send message and wait for response”** (Gmail, operacion sendAndWait), ubicado entre la clasificacion de IA y el contacto real al huesped.

Parametro	Valor
Destinatario de la aprobacion	elisabethelfi@gmail.com (la operadora, no el huesped)
Tipo de aprobacion	"double" (confirmacion en dos pasos)
Contenido del email	Nombre, mail, perfil de estadia, prioridad y el resumen + respuesta sugerida por la IA
Comportamiento	El flujo se detiene ("wait") hasta recibir la respuesta antes de continuar

Esto cumple el requisito de evitar el “efecto metralleta”: **ningun mensaje llega al huesped sin que un humano revise y apruebe explícitamente** lo que la IA sugirió. Recien cuando el nodo If confirma approved == true, el flujo dispara “Send a message” hacia el huesped real; si la respuesta es negativa, el lead se marca “Descartado” sin que se envíe ninguna comunicacion externa.

Configuracion relevada directamente del blueprint JSON exportado del flujo n8n en uso (AlmaAlquiler), al 10/09/2026.